

git log, 되돌리기등

- 02_깃으로 버전관리하기.pdf의 14페이지부터 ~

1 git log

pdf자료 02_깃으로 버전 관리하기.pdf 의 14p-16p

- 전체 커밋 히스토리 보기 `git log`

마지막 (END) q

```
commit db518dde29a31e8f03d1142983fe60ba712cd329 (HEAD -> main, origin/main)
Author: 부성순 <bakpak@empas.com>
Date:   Wed Dec 3 17:23:27 2025 +0900

    결측치 추가

commit ed304479ab7374ba310649932169730d0a81ba87
Author: 부성순 <bakpak@empas.com>
Date:   Wed Dec 3 16:59:06 2025 +0900

    기술통계 추가

commit 6804d808373a8a11525eb3fe9a7d39aa34659a14
Author: 부성순 <bakpak@empas.com>
Date:   Wed Dec 3 16:53:22 2025 +0900

    first commit
(END)
```

- 한 줄로 깔끔하게 히스토리 보기 (제일 많이 씀) `git log --oneline`

```
db518dd (HEAD -> main, origin/main) 결측치 추가
ed30447 기술통계 추가
6804d80 first commit
(END)
```

- 그래프 형태로 브랜치 흐름까지 보기 `git log --oneline --graph --all`

- 브랜치가 어떻게 나뉘고 합쳐졌는지 시각적으로 보여줌./ 현재는 브랜치가 main만
존

```
* db518dd (HEAD -> main, origin/main) 결측치추가
* ed30447 기술 통계 추가
* 6804d80 first commit
(END)
```

- push안한 커밋보기: `git log origin/main..HEAD --oneline`
- 이미 push된 히스토리 보기: `git log origin/main --oneline`
- 최근 N개만 보기: `git log -3 --oneline`
- github에서도 push이력볼수 있음

python Public

Pin Watch 0 Fork 0 Star 0

main Go to file + Code About

pjm9889 결측치추가 db518dd · 8 minutes ago

.ipynb_checkpoints	first commit	38 minutes ago
pandas	결측치추가	8 minutes ago
untitled.py	first commit	38 minutes ago

Activity 0 stars 0 watching 0 forks

- 수정했지만 push하지 않은 자료 (github main자료와 비교) , app.py의 마지막에 추가저장

```
print("codeit python")
```

- wsl화면에서 `git diff`

현재 파일 vs 마지막 커밋(HEAD) 사이의 차이를 보여줌

ipynb는 자동으로 checkpoint가 생성됨으로 .ipynb하고 체크포인트 둘다 비교

출력형태 `diff --git 이전버전/pandas/app.py 수정된버전/pandas/app.py`

- 이미 commit 했지만, 다시 이전 버전 파일을 가져오고 싶으면:

`git restore --source=HEAD~1 ./pandas/app.py`

후 app.py를 보면 마지막 print문이 제거

상황	<code>git restore</code> 의미 있음?	설명
commit 전 (unstaged/staged)	○ 매우 의미 있음	작업 중인 파일을 이전 커밋 상태로 되돌림
commit 후 (이미 commit 완료)	○ 의미 있음	이전 커밋 상태로 "파일을 복구"하지만, 히스토리는 유지됨
commit 자체를 취소하고 싶을 때	✗ 의미 없음	이 경우 <code>git reset</code> 을 사용해야 함

- 파일은 삭제해도 `git log`와 `git show`에 기록이 남아 있으므로 commit기록삭제나, 또는 log자체를 싹 지워야 하기도함

- `git log --oneline` 해보면 여전히 print문 추가했던 작업기록은 보임

```
6eb4a09 (HEAD → main) print문추가
db518dd (origin/main) 결측치추가
ed30447 기술통계추가
6804d80 first commit
```

- 직전 커밋 삭제: `git reset --hard HEAD~1`

```
userbu@DESKTOP-96N9HJ3:~/python$ git reset --hard HEAD~1
HEAD is now at db518dd 결측치추가
```

- `git log --oneline` 해서 다시 확인
- `git log` 전체를 지우고, 현재 상태만 남기는 가장 쉬운 방법

```
git checkout --orphan new-history
git add .
git commit -m "initial commit (history removed)"
git branch -D main
git branch -m main
git push -f origin main
```

명령어	의미	결과/효과
git checkout --orphan new-history	기존 히스토리와 단절된 새로운 브랜치 생성	히스토리 0개인 새 브랜치 시작 (파일은 그대로 유지)
git add .	현재 폴더의 모든 파일 stage	새 브랜치에 첫 커밋 준비
git commit -m "initial commit (history removed)"	새 브랜치에서 첫 커밋 생성	히스토리가 1개만 있는 상태가 됨
git branch -D main	기존 main 브랜치 강제 삭제	기존 히스토리 포함된 main 브랜치 제거
git branch -m main	현재 브랜치(new-history)의 이름을 main으로 변경	새로 만든 히스토리를 main으로 설정
git push -f origin main	원격(GitHub) main을 강제로 덮어쓰기	GitHub의 기존 히스토리도 완전히 삭제됨